

DESIGN & CALCULATIONS

Truth Table:

Hex Digit	Inputs				Outputs							
Digit	D3	D2	D1	D0	Sg	Sf	Se	Sd	Sc	Sb	Sa	S In Hex
0	0	0	0	0	1	0	0	0	0	0	0	40
1	0	0	0	1	1	1	1	1	0	0	1	79
2	0	0	1	0	0	1	0	0	1	0	0	24
3	0	0	1	1	0	1	1	0	0	0	0	30
4	0	1	0	0	0	0	1	1	0	0	1	19
5	0	1	0	1	0	0	1	0	0	1	0	12
6	0	1	1	0	0	0	0	0	0	1	0	2
7	0	1	1	1	1	1	1	1	0	0	0	78
8	1	0	0	0	0	0	0	0	0	0	0	0
9	1	0	0	1	0	0	1	1	0	0	0	18
A	1	0	1	0	0	0	0	1	0	0	0	8
B	1	0	1	1	0	0	0	0	0	1	1	3
C	1	1	0	0	0	1	0	0	1	1	1	27
D	1	1	0	1	0	1	0	0	0	0	1	21
E	1	1	1	0	0	0	0	0	1	1	0	6
F	1	1	1	1	0	0	0	1	1	1	0	E

Circuit/ Block Diagrams + Logic Equations:

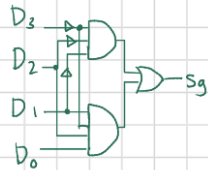
LAB 5: 7-SEGMENT DECODER

* input [3:0] D

Sg:

32\10	00	01	11	10
00	1	1	0	0
01	0	0	1	0
11	0	0	0	0
10	0	0	0	0

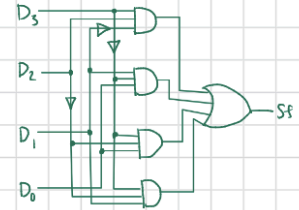
$$S_g = \bar{3}\bar{2}\bar{1} + \bar{3}210$$



Sf:

32\10	00	01	11	10
00	0	1	1	1
01	0	0	1	0
11	1	1	0	0
10	0	0	0	0

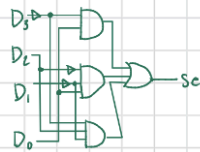
$$S_f = 32\bar{1} + \bar{3}10 + \bar{3}\bar{2}0 + \bar{3}\bar{2}1$$



Sc:

32\10	00	01	11	10
00	0	1	1	0
01	1	1	1	0
11	0	0	0	0
10	0	1	0	0

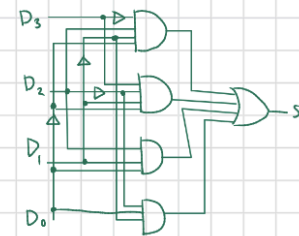
$$S_c = \bar{2}10 + \bar{3}0 + \bar{3}\bar{2}1$$



Sd:

32\10	00	01	11	10
00	0	1	0	0
01	1	0	1	0
11	0	0	1	0
10	0	1	0	1

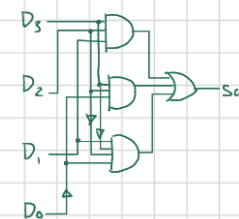
$$S_d = \bar{3}\bar{2}\bar{1}0 + \bar{3}\bar{2}1\bar{0} + 210 + \bar{2}1\bar{0}$$



Se:

32\10	00	01	11	10
00	0	0	0	1
01	0	0	0	0
11	1	0	1	1
10	0	0	0	0

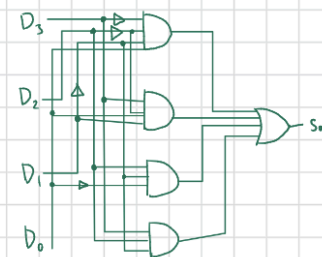
$$S_e = 321 + 32\bar{0} + \bar{3}\bar{2}1\bar{0}$$



Sn:

32\10	00	01	11	10
00	0	1	0	0
01	1	0	0	0
11	1	0	0	0
10	0	0	1	0

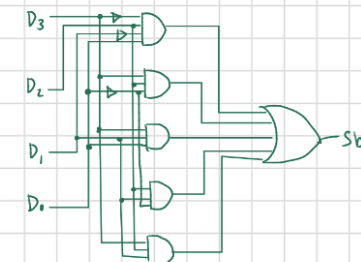
$$S_n = \bar{3}\bar{2}10 + \bar{3}\bar{2}1\bar{0} + 21\bar{0} + 32\bar{1}$$



Sb:

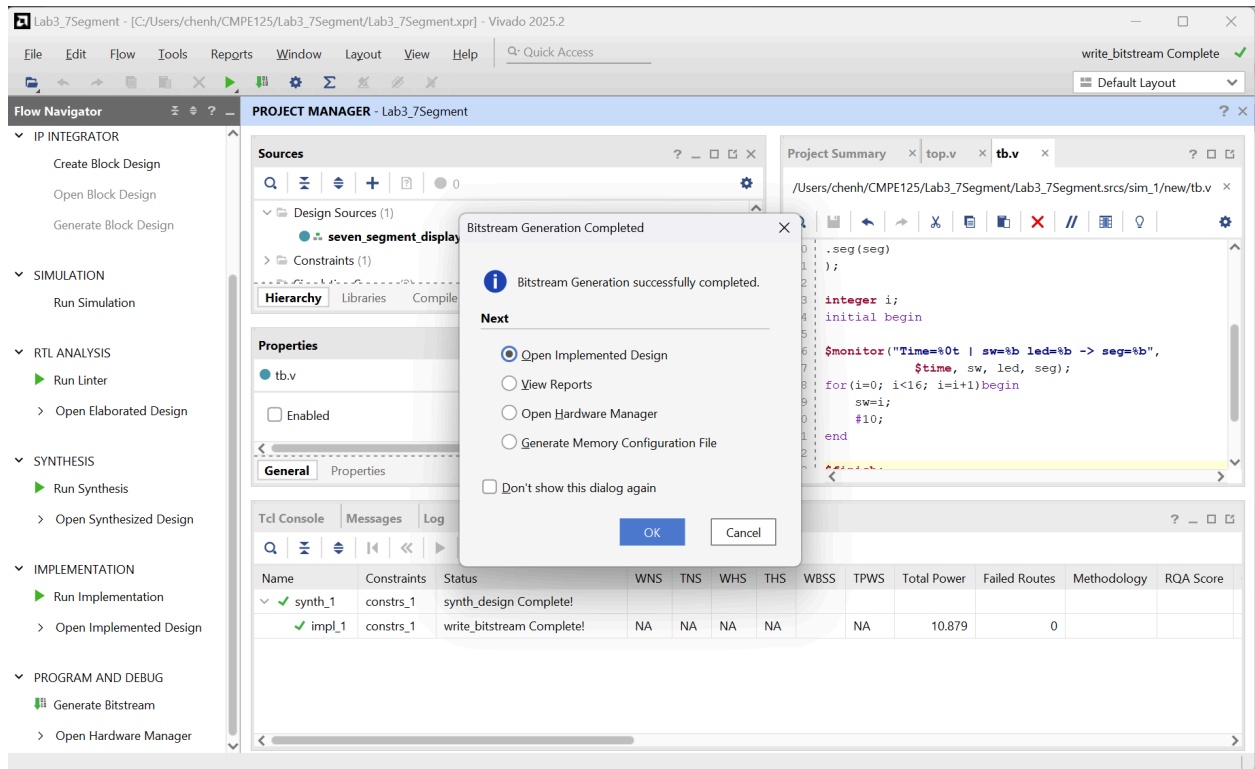
32\10	00	01	11	10
00	0	0	0	0
01	0	1	0	1
11	1	0	1	1
10	0	0	1	0

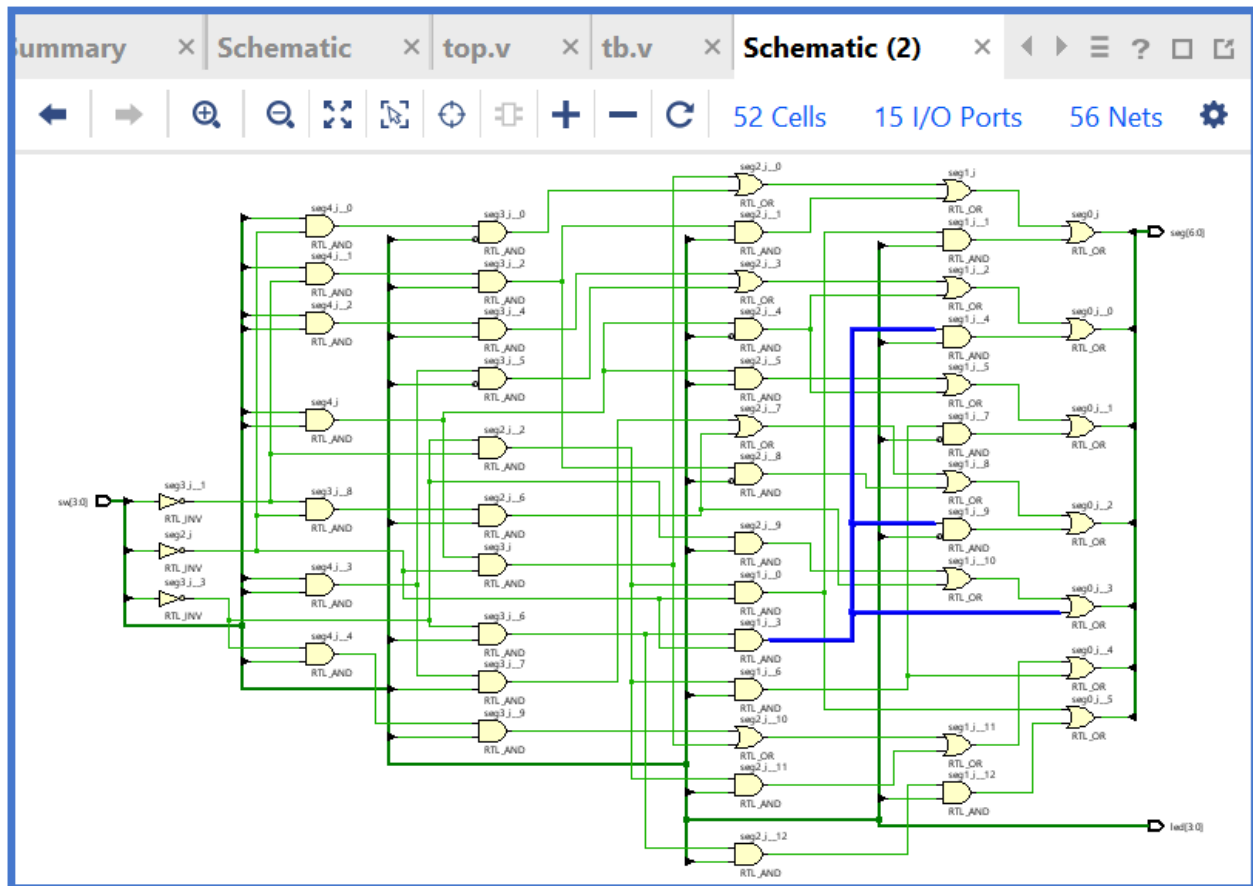
$$S_b = \bar{3}\bar{2}1\bar{0} + 32\bar{0} + 810 + 21\bar{0} + 32\bar{1}$$



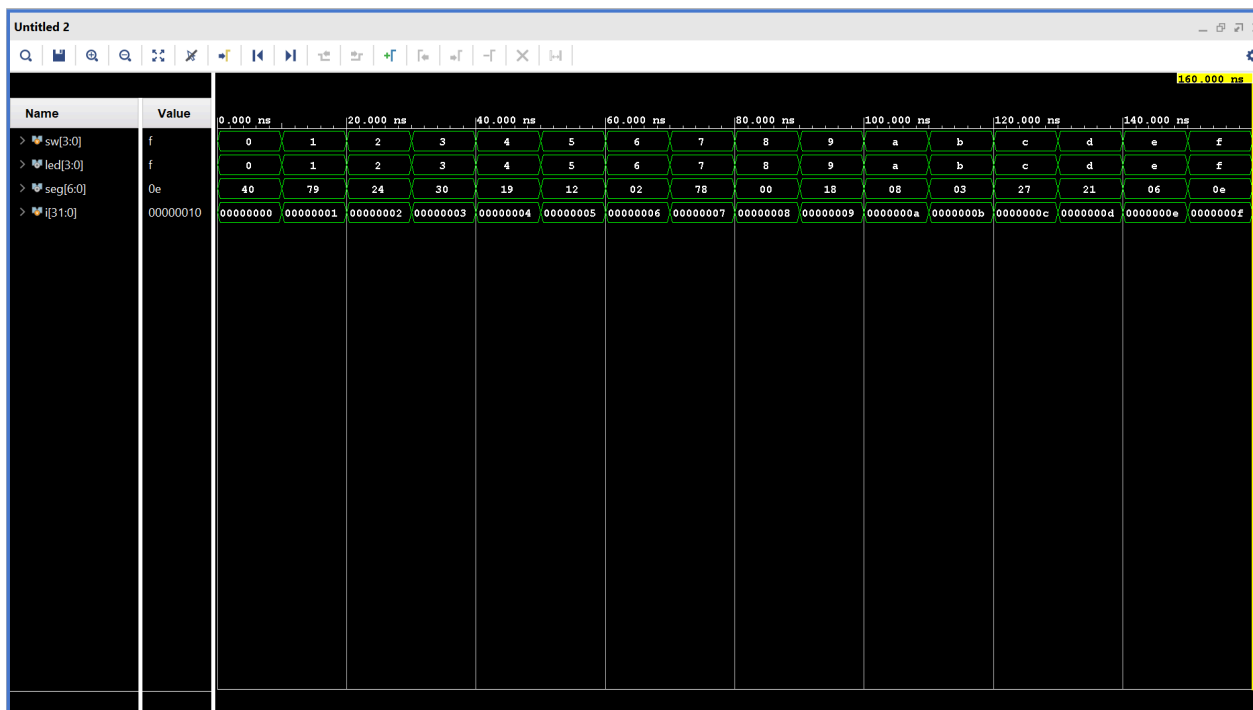
SCREENSHOTS

Compilation:





Simulation Waveforms:





SUMMARY

The objective of this lab was to design a low-active seven-segment display decoder that takes in a four-bit binary bus that is converted to hex and displayed via led on the FPGA board. To accomplish this, each segment of the LED display was assigned a letter from Sa...Sg. A truth table was then implemented to show the state of each of the 7 segments corresponding to the input bus values [3:0]D (assigned to switches), according to the provided diagrams on the lab manual. LED displays above the respective switches were also enabled to indicate when they are on. From there, the optimized boolean logic equations were extracted from the individual K-maps made for each segment, and the behavioral verilog code was written. A testbench was also written to cycle through all 16 input combinations, and the segment bus's hex value at each input matched that on the truth table. The output display on the FPGA board matched the expected display diagrams.

VERILOG CODE

Top.v:

```

module seven_segment_display(
//led0123 corresponds to D bus for visibility/ convenience
//Sg = seg[6]...Sa = seg[0]
//use input wire because, module is not the one driving it
    input wire [3:0]sw,
    output wire [3:0]led,
    output wire[6:0]seg
);
assign led = sw;
```

```

assign seg[6] = (~sw[3]&~sw[2]&~sw[1]) | (~sw[3]&sw[2]&sw[1]&sw[0]);
assign seg[5] = (~sw[3]&sw[1]&sw[0]) | (sw[3]&sw[2]&~sw[1]) |
(~sw[3]&~sw[2]&sw[0])|(~sw[3]&~sw[2]&sw[1]);
assign seg[4] = (~sw[3]&sw[0])|(~sw[2]&~sw[1]&sw[0])|
(~sw[3]&sw[2]&~sw[1]);
assign seg[3] = (sw[2]&sw[1]&sw[0])|(~sw[2]&~sw[1]&sw[0])|
(sw[3]&~sw[2]&sw[1]&~sw[0])|(~sw[3]&sw[2]&~sw[1]&~sw[0]);
assign seg[2] = (sw[3]&sw[2]&sw[1])|(sw[3]&sw[2]&~sw[0])|
(~sw[3]&~sw[2]&sw[1]&~sw[0]);
assign seg[1] = (sw[3]&sw[1]&sw[0])|(sw[2]&sw[1]&~sw[0])|(sw[3]&sw[2]&~sw[0])|
(~sw[3]&sw[2]&~sw[1]&sw[0]);
assign seg[0] = (sw[3]&sw[2]&~sw[1])|(sw[2]&~sw[1]&~sw[0])|
(sw[3]&~sw[2]&sw[1]&sw[0])|(~sw[3]&~sw[2]&~sw[1]&sw[0]);

endmodule

```

Tb.v:

```

`timescale 1ns/1ps
module tb;
reg [3:0] sw;
wire [3:0] led;
wire[6:0] seg;

seven_segment_display dut(
.sw(sw),
.led(led),
.seg(seg)
);

integer i;
initial begin

$monitor("Time=%0t | sw=%b led=%b -> seg=%b",
$time, sw, led, seg);
for(i=0; i<16; i=i+1)begin
sw=i;
#10;
end
end

```

```
$finish;  
end  
  
endmodule
```

TROUBLESHOOTING

I had a bit of trouble generating the simulated waveform due to some syntax errors in the testbench, as I declared the testbench wires and registers “input” and “output”—these keywords are only supposed to be used in the module. I also tried to understand the difference between “wire” and “reg”: Reg holds values and is required for variables that are assigned inside a procedural block (always, init). Wires are driven continuously and cannot store values. In the testbench, signals that you drive use reg and signals that the dut drives need wire. We assign wire to the module’s sw bus, but assign reg to the testbench’s sw bus because the module expects the buses to be driven from the outside (FPGA board), while the testbench involves a procedural “initial” block in which a for loop assigns different values to the sw bus.

FPGA BOARD OUTPUT

